개발 워크스테이션 구축 미션

초등학생도 똑같이 따라하는 도커(Docker) 웹서버 만들기

터미널 · 권한 · Docker · Dockerfile · 포트 · 볼륨 · Git · GitHub 전 과정

이 책을 다 따라하면 이런 게 생겨요

① 내가 만든 웹사이트가 들어있는 **도커 이미지**
② 브라우저에서 열리는 **나만의 웹 서버 주소** (http://localhost:8081)
③ 파일을 고치면 바로 바뀌는 **바인드 마운트**
④ 컨테이너를 지워도 안 없어지는 **도커 볼륨**
⑤ 선생님께 링크만 보내면 되는 **GitHub 저장소 + README**

읽는 방법 (아주 중요!)

· **검은색 상자**는 내가 **직접 쳐야 하는 명령어**예요. 한 줄씩, 치고 Enter!
· **회색 상자**는 컴퓨터가 **대답해 주는 글**이에요. 비슷하게 나오면 성공!
· **초록 상자**는 "이렇게 되면 성공", **빨간 상자**는 "이러면 틀린 거예요".
· 건너뛰지 말고 **STEP 1부터 순서대로** 하세요. 순서를 지키면 에러가 안 납니다.

✅ **이 책의 모든 명령은 실제로 실행해서 확인했습니다**
2026-11-03, Windows 11 Pro + **Docker 29.9.2** (Docker Desktop 4.97.0, WSL2) 환경에서 40개 명령을 그대로 실행한 **진짜 출력**을 실었습니다. 에러 메시지도 일부러 재현해서 받아 적은 것이에요.

작성일 2026-11-03 · Windows 11 + PowerShell + Docker Desktop 4.97.0 기준

0 먼저 전체 그림부터 보기

이 미션이 뭐예요?
"내 컴퓨터를 **개발자용 작업대(워크스테이션)**로 꾸미고, 그 증거를 GitHub에 올리기"가 전부예요. 웹사이트를 만들어서 **도커라는 상자**에 담고, 그 상자를 실행해서 브라우저로 구경한 다음, 한 일을 **README.md라는 설명서**에 적어 인터넷(GitHub)에 올리면 끝!

도커가 뭔지 1분 설명

컴퓨터에 프로그램을 깔면 "내 컴퓨터에선 되는데 친구 컴퓨터에선 안 돼요" 하는 일이 자주 생겨요. 그래서 **프로그램 + 필요한 재료 전부**를 **도시락통**에 통째로 담는 기술이 도커입니다.

도커 말	쉬운 말	설명
이미지 (image)	도시락 **레시피 + 재료 세트**	아직 안 움직이는 '설계도'. `my-web-server` 가 내가 만든 이미지예요.
컨테이너 (container)	진짜 **만들어진 도시락**	이미지를 실행하면 생기는 '움직이는 것'. `my-site` 가 그것.
Dockerfile	레시피를 적은 **종이**	"어떤 재료로 어떻게 만들어라"를 적은 글자 파일.
포트 매핑 (-p)	도시락통의 **빨대 구멍**	상자 안(80번)을 내 컴퓨터(8081번)에 연결해서 들여다보는 통로.
바인드 마운트	상자에 뚫은 **창문**	내 컴퓨터 폴더를 상자 안에서 바로 보게 함. 고치면 즉시 반영!
볼륨 (volume)	따로 두는 **보관함**	상자를 버려도 안에 넣은 물건이 남아있음.

우리가 지나갈 길 (이 순서대로만 하세요)

STEP 1 터미널 열기 · **STEP 2** 권한 확인 · **STEP 3** 도커 점검 · **STEP 4** 폴더·웹페이지 만들기 · **STEP 5** Dockerfile 쓰기 · **STEP 6** 빌드 & 실행(포트 8081) · **STEP 7** 운영 명령 로그 모으기 · **STEP 8** 바인드 마운트(8083) · **STEP 9** 볼륨 영속성(8082) · **STEP 10** Git 설정 · **STEP 11** VSCode·GitHub 올리기 · **STEP 12** README 쓰고 제출

우리가 쓸 폴더와 이름 (끝까지 똑같이 쓰기!)

항목	이름	설명
작업 폴더	`C:\study\my-web-server`	모든 작업은 여기서 해요
웹페이지 폴더	`app\index.html`	내가 만든 홈페이지
이미지 이름	`my-web-server`	Dockerfile로 구운 내 이미지
컨테이너 이름 / 포트	`my-site` / **8081**	메인 제출용 웹 서버
연습용 nginx	`my-final-web` / **8080**	이미 만들어 둔 것 (그대로 두기)
볼륨 실습	`vol-test` / **8082**	STEP 9에서 사용
바인드 마운트 실습	`live-site` / **8083**	STEP 8에서 사용

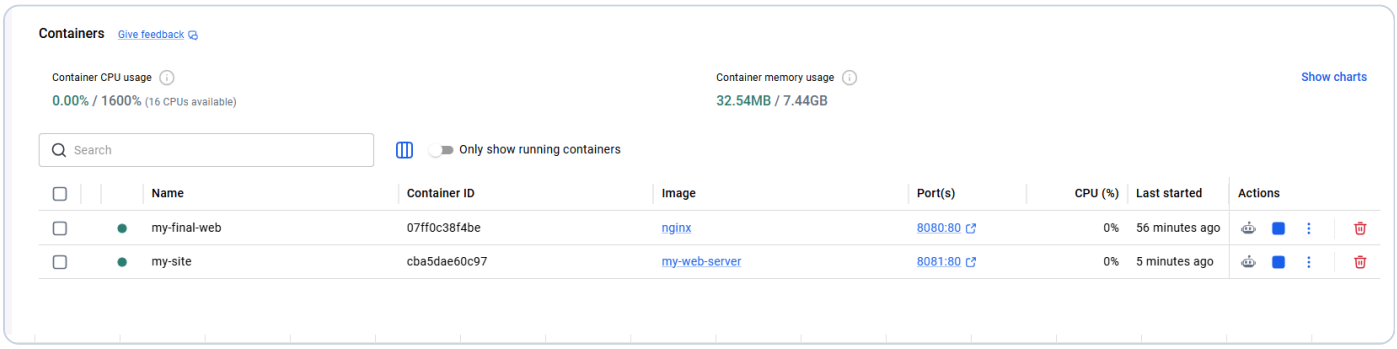
> **포트 번호는 왜 다 달라요?** 한 번호에는 한 개만 들어갈 수 있어요. 8080에 이미 `my-final-web`이 살고 있으니, 새 컨테이너는 8081·8082·8083처럼 **빈 번호**를 써야 충돌 에러가 안 납니다.

지금까지 해 놓은 것 (이미 완료!)

아래는 지금까지 실제로 성공한 화면이에요. 여기까지 되어 있으면 STEP 4부터 이어서 해도 됩니다.

```
PS C:\study> docker run -d -p 8081:80 --name my-site my-web-server
cba5dae60c97400308e4eeac70491fae5358a90dedb1a90018276d5ffc015d91
PS C:\study> docker images
                                                                  i Info →  U  In Use
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
my-web-server:latest   230de1b6fe70       241MB         63.9MB    U
nginx:latest           5ab5b0b01097       243MB         66.8MB    U
PS C:\study> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS
               NAMES
cba5dae60c97   my-web-server   "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:8081->80/tcp, [::]:
8081->80/tcp   my-site
07ff0c38f4be   nginx          "/docker-entrypoint.…"   54 minutes ago  Up 54 minutes  0.0.0.0:8080->80/tcp, [::]:
8080->80/tcp   my-final-web
PS C:\study>
```

▲ `docker images`에 my-web-server·nginx 두 개, `docker ps`에 my-site(8081)·my-final-web(8080) 두 개가 Up 상태



▲ Docker Desktop 화면에서도 두 컨테이너가 초록불(실행 중)

1 터미널(PowerShell) 열고 길 찾기

목표 : 검은 창(터미널)을 열고, 내가 지금 **어느 폴더에 서 있는지** 알고, 폴더를 만들고 이동할 수 있다.

터미널이 뭐예요? 마우스로 더블클릭하는 대신 **글자로 컴퓨터에게 심부름을 시키는 창**이에요. 개발자는 마우스보다 이게 훨씬 빠르기 때문에 꼭 익혀야 해요.

1-1. 터미널 열기

1. 키보드에서 **Windows 키**를 누릅니다.
2. **powershell** 이라고 칩니다.
3. **Windows PowerShell** 을 클릭해서 엽니다. (검은색/파란색 창이 뜨면 성공)

1-2. 기본 명령 5개만 외우기

명령어	무슨 뜻?	예시
`pwd`	나 지금 어디 있어?	`pwd`
`cd 폴더이름`	그 폴더로 들어가기	`cd C:\study`
`mkdir 이름`	새 폴더 만들기	`mkdir app`
`dir`	이 폴더에 뭐가 있나 보기	`dir`
`type 파일`	파일 내용 읽기	`type Dockerfile`

cd .. 는 "한 칸 위 폴더로 나가기"예요. **Tab 키**를 누르면 폴더 이름을 자동으로 완성해 줘서 오타가 안 납니다.

1-3. 작업 폴더 만들고 들어가기 (실제로 해보기)

터미널에 한 줄씩 입력

```
mkdir C:\study\my-web-server\app
cd C:\study\my-web-server
pwd
dir
```

이렇게 나오면 정상 (실제 실행 결과)

```
Path
----
C:\study\my-web-server

Mode   LastWriteTime          Length Name
----   -------------          ------ ----
d----- 2026-11-02 오후 10:28      app
```

성공 확인 : `pwd` 결과가 **C:₩study₩my-web-server** 이고, `dir` 에 **app** 폴더가 보이면 OK!

자주 하는 실수 : `mkdir` 로 만든 폴더 이름에 **띄어쓰기**나 **한글**을 쓰면 나중에 도커에서 에러가 납니다. 꼭 영어 소문자와 **-** 만 쓰세요.

2 권한(Permission) 확인하기

목표 : 내가 누구인지(`whoami`), 파일에 누가 손댈 수 있는지(`icacls`), 관리자 권한이 왜 필요한지 확인한다.

권한이 뭐예요? 학교 교실 열쇠 같은 거예요. 아무나 못 들어가게 컴퓨터가 **"누가 읽어도 되고, 누가 고쳐도 되는지"**를 정해 둡니다. 도커를 설치하거나 포트를 여는 일은 **힘이 센 권한(관리자)**이 필요할 때가 있어요.

2-1. 나는 누구? 내 컴퓨터 이름은?

입력

```
whoami
```

실제 출력 예시

```
desktop-1eet513\lovehjlee
```

컴퓨터이름₩내계정이름 형식으로 나와요. 이게 "지금 컴퓨터를 쓰고 있는 사람"입니다.

2-2. 파일 권한 들여다보기

입력 (C:₩study₩my-web-server 안에서)

```
icacls Dockerfile
```

실제 출력 예시

```
Dockerfile BUILTIN\Administrators:(I)(F)
           NT AUTHORITY\SYSTEM:(I)(F)
           BUILTIN\Users:(I)(RX)
           NT AUTHORITY\Authenticated Users:(I)(M)

Successfully processed 1 files; Failed processing 0 files
```

글자	뜻	쉬운 설명
F	Full (모두)	읽고 고치고 지우고 다 할 수 있음
M	Modify (수정)	읽고 고칠 수 있음
RX	Read & eXecute	읽고 실행만 가능, 고치지는 못함
(I)	Inherited	위 폴더에서 물려받은 권한

성공 확인 : 위처럼 줄들이 나오고 **Successfully processed 1 files** 가 보이면 OK.

2-3. 관리자 권한으로 터미널 여는 법

1. Windows 키 → **powershell** 입력
2. 마우스 **오른쪽 클릭** → **관리자 권한으로 실행**
3. "이 앱이 변경하도록 허용할까요?" → **예**
4. 창 제목에 **관리자:** 가 붙으면 성공

스크립트 실행 정책 확인 (참고용)

```
Get-ExecutionPolicy
```

실제 출력 예시

```
Bypass
```

조심! 권한을 바꾸는 명령(`icacls /grant`, `Set-ExecutionPolicy`)은 이 미션에서 **필요 없어요**. 확인만 하고 넘어가세요. 잘못 바꾸면 컴퓨터가 이상해질 수 있습니다.

도커와 권한 : Docker Desktop을 설치하면 내 계정이 **docker-users** 라는 그룹에 들어가요. 그래서 관리자 창이 아니어도 `docker` 명령을 쓸 수 있는 거예요.

3 도커가 잘 깔렸는지 점검하기

목표 : `docker --version`, `docker info`, `docker run hello-world` 세 가지로 도커가 살아있는지 확인한다.

3-0. 먼저 Docker Desktop을 켜세요

작업표시줄 오른쪽 아래 **고래 아이콘** 🐳 이 있어야 해요. 없으면 Windows 키 → **Docker Desktop** 실행 → 고래가 **초록색/흰색으로 멈춤(Engine running)** 될 때까지 1~2분 기다립니다.

이거 안 하면 무조건 에러! Docker Desktop이 꺼져 있으면 아래 명령이 전부 `error during connect ... dockerDesktopLinuxEngine` 로 실패합니다.

3-1. 버전 확인

입력

```
docker --version
docker compose version
git --version
```

실제 실행 결과 (숫자는 조금 달라도 됩니다)

```
Docker version 29.9.2, build a82e528
Docker Compose version v5.6.1
git version 2.54.0.windows.1
```

성공 확인 : 세 줄 모두 **version** 글자가 보이면 OK. 이 결과를 그대로 복사해서 나중에 README에 붙여 넣을 거예요.

3-2. 도커 상태 자세히 보기

입력

```
docker info
```

실제 실행 결과 중 중요한 부분만

```
Client:
 Version:    29.9.2
 Context:    desktop-linux

Server:
 Containers: 2
  Running: 2
  Paused: 0
  Stopped: 0
 Images: 3
 Server Version: 29.9.2
 Storage Driver: overlayfs
 Kernel Version: 6.18.54.1-microsoft-standard-WSL2
 Operating System: Docker Desktop
 OSType: linux
 CPUs: 16
 Total Memory: 15.18GiB
 Name: docker-desktop
```

> 글자가 너무 많이 나와서 놀라지 마세요. **Containers**(상자 개수)와 **Images**(레시피 개수)만 보면 됩니다.

3-3. 진짜로 컨테이너 한 번 돌려보기

입력

```
docker run hello-world
```

실제 실행 결과

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

> 맨 윗줄 **Unable to find image ... locally** 는 에러가 아니에요! "내 컴퓨터에 없으니 인터넷에서 받아올게" 라는 뜻이고, 바로 아래 **Pulling / Pull complete** 가 내려받는 과정입니다.

> **성공 확인** : **Hello from Docker!** 가 보이면 도커 설치 완벽!

4 내 웹페이지(소스코드) 만들기

목표 : `C:\study\my-web-server\app\index.html` 파일에 내 홈페이지를 만든다.

index.html 이 뭐예요? 웹 서버는 손님이 오면 **index.html** 이라는 파일을 제일 먼저 보여줘요. 그래서 홈페이지 첫 화면의 이름은 항상 index.html 입니다.

4-1. 메모장으로 파일 만들기 (가장 쉬운 방법)

1. 파일 탐색기에서 `C:\study\my-web-server\app` 폴더를 엽니다.
2. 빈 곳에 오른쪽 클릭 → **새로 만들기 → 텍스트 문서**
3. 이름을 **index.html** 로 바꿉니다. (**.txt 가 남아있지 않게** 주의!)
4. 메모장으로 열고 아래 코드를 그대로 적습니다.
5. **파일 → 다른 이름으로 저장 → 인코딩을 UTF-8** 로 고르고 저장합니다. ← **한글이 안 깨지려면 꼭!**

VSCode를 쓴다면 더 쉬워요. VSCode에서 폴더를 열고 **새 파일** 아이콘 → `index.html` 입력 → 코드 붙여넣기 → **Ctrl+S**.

4-2. index.html 전체 코드 (그대로 따라 쓰기)

C:₩study₩my-web-server₩app₩index.html

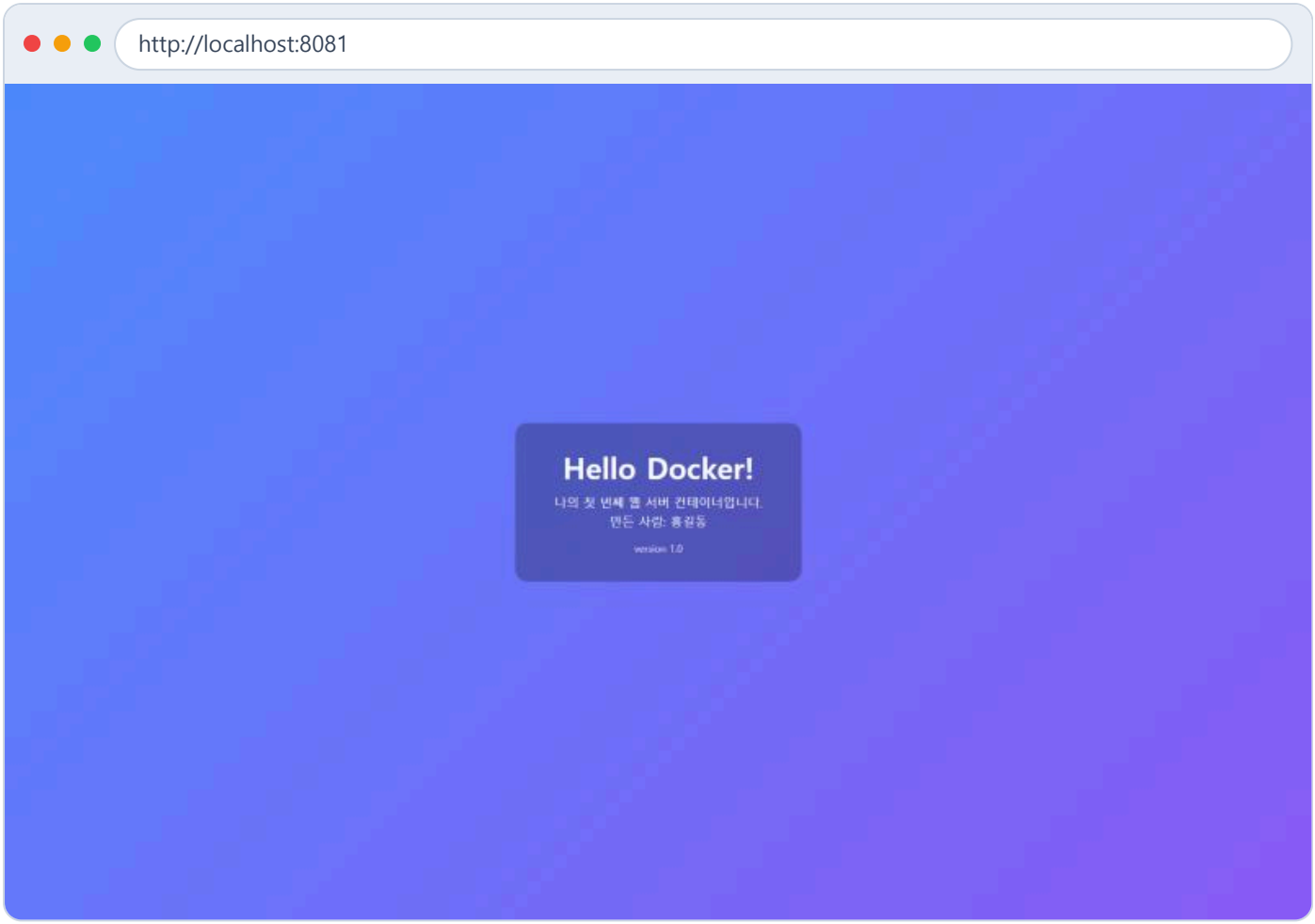
```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>나의 첫 웹 서버</title>
  <style>
    body {
      margin: 0; height: 100vh;
      display: flex; flex-direction: column;
      align-items: center; justify-content: center;
      font-family: "Malgun Gothic", sans-serif;
      background: linear-gradient(135deg, #4f8cff, #8b5cf6);
      color: #fff; text-align: center;
    }
    h1 { font-size: 48px; margin: 0 0 12px; }
    p  { font-size: 20px; margin: 4px; }
    .box { background: rgba(0,0,0,0.25); padding: 40px 60px; border-radius: 20px; }
    .ver { font-size: 16px; opacity: 0.8; margin-top: 20px; }
  </style>
</head>
<body>
  <div class="box">
    <h1>Hello Docker!</h1>
    <p>나의 첫 번째 웹 서버 컨테이너입니다.</p>
    <p>만든 사람: 홍길동</p>      <!-- ← 내 이름으로 바꾸기! -->
    <p class="ver">version 1.0</p>
  </div>
</body>
</html>
```

4-3. 잘 만들어졌는지 확인

입력

```
cd C:\study\my-web-server
dir app
type app\index.html
```

성공 확인 : `dir app` 에 **index.html** 이 보이고, `type` 했을 때 방금 쓴 코드가 주르륵 나오면 OK. 파일을 더블클릭해서 브라우저로 열어 봐도 좋아요(아직 도커는 아니지만 모양은 똑같습니다).



▲ 완성되면 보게 될 화면 (파란-보라 배경 + Hello Docker!) — **실제 컨테이너에서 찍은 화면**

5 Dockerfile(레시피) 쓰기

목표 : `C:\study\my-web-server\Dockerfile` 을 만들고 각 줄이 무슨 뜻인지 이해한다.

파일 이름 주의! Dockerfile 입니다. 대문자 D, 확장자 **없음**. `Dockerfile.txt`, `dockerfile.html` 로 저장하면 빌드가 안 됩니다. 메모장에서 저장할 때 **파일 형식: 모든 파일(*.*)** 을 고르고 이름을 **Dockerfile** 로 쓰세요.

5-1. Dockerfile 전체 내용

C:₩study₩my-web-server₩Dockerfile

```
# 1) 바탕이 되는 이미지: 공식 nginx 웹 서버
FROM nginx:latest

# 2) 이 이미지를 만든 사람 정보
LABEL maintainer="student@example.com"
LABEL description="My first web server built with Dockerfile"

# 3) 내 웹 페이지를 nginx 기본 폴더로 복사
COPY app/ /usr/share/nginx/html/

# 4) 컨테이너가 사용하는 포트를 알려줌 (문서 역할)
EXPOSE 80

# 5) 컨테이너가 켜질 때 실행할 명령
CMD ["nginx", "-g", "daemon off;"]
```

5-2. 한 줄씩 무슨 뜻일까?

명령	쉬운 설명
FROM nginx:latest	맨바닥부터 만들면 너무 힘드니, **이미 완성된 웹 서버(nginx)**를 바탕으로 깔고 시작해요. "식빵 사 와서 샌드위치 만들기"와 같아요.
LABEL	이름표예요. 누가 만들었는지 적어 둡니다. 없어도 되지만 과제에서는 적어주면 좋아요.
COPY app/ /usr/share/nginx/html/	내 컴퓨터의 **app 폴더**를 상자 안 **/usr/share/nginx/html/** 로 복사. nginx는 이 폴더를 손님에게 보여줍니다.
EXPOSE 80	"이 상자는 80번 구멍을 씁니다"라는 **안내문**. (실제 연결은 실행할 때 -p 로 해요)
CMD [...]	상자를 켤 때 자동으로 돌릴 명령. **daemon off;** 는 "뒤로 숨지 말고 계속 앞에서 일해"라는 뜻인데, 이게 없으면 컨테이너가 켜지자마자 꺼져요.

성공 확인 : `dir` 했을 때 **Dockerfile** 이 확장자 없이 보이고, 크기가 0이 아니면 OK.

실제 확인 결과

```
Mode   LastWriteTime          Length Name
----   -------------          ------ ----
d----- 2026-11-02 오후 10:28        app
-a---- 2026-11-02 오후 10:28    139  .gitignore
-a---- 2026-11-02 오후 10:28    476  Dockerfile
```

6 이미지 굽기(build) → 실행(run) → 브라우저 접속

목표 : 레시피(Dockerfile)로 이미지를 굽고, 컨테이너를 8081 포트로 띄워 브라우저에서 본다.

6-1. 이미지 빌드

입력 (반드시 Dockerfile 이 있는 폴더에서!)

```
cd C:\study\my-web-server
docker build -t my-web-server .
```

조각	뜻
`build`	레시피로 이미지를 구워라
`-t my-web-server`	구운 이미지 **이름표(tag)**를 my-web-server 로 붙여라
`.` (점)	**지금 이 폴더**에 있는 Dockerfile을 써라 ← **점 빼먹으면 에러!**

실제 실행 결과 (마지막 두 줄이 중요)

```
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 515B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/nginx:latest
#2 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 1.06kB done
#4 DONE 0.0s

#5 [1/2] FROM docker.io/library/nginx:latest@sha256:5ab5b0b01097...
#5 DONE 1.2s

#6 [2/2] COPY app/ /usr/share/nginx/html/
#6 DONE 0.2s

#7 exporting to image
#7 naming to docker.io/library/my-web-server:latest done
#7 DONE 0.1s
```

성공 확인 : 마지막에 **naming to docker.io/library/my-web-server:latest done** 이 보이면 성공!
#6 [2/2] COPY app/ 줄은 내 웹페이지가 상자 안으로 잘 복사됐다는 뜻이에요.

6-2. 이미지가 생겼는지 확인

입력

```
docker images
```

실제 실행 결과

```
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest     c567b231f656       25.9kB         9.49kB
my-web-server:latest   760997405362        241MB         63.9MB
nginx:latest           5ab5b0b01097        243MB         66.8MB   U
```

my-web-server 가 241MB나 돼요? 바탕으로 쓴 nginx(243MB)가 통째로 들어있기 때문이에요. 맨 오른쪽 **U** 는 "지금 사용 중(Used)"이라는 표시입니다.

6-3. 컨테이너 실행 (포트 매핑!)

입력

```
docker run -d -p 8081:80 --name my-site my-web-server
```

조각	뜻
`-d`	뒤에서 몰래 돌려라(detached). 터미널이 안 멈춰요.
`-p 8081:80`	**왼쪽=내 컴퓨터 8081번, 오른쪽=상자 안 80번**. 8081로 들어가면 상자 80번으로 연결!
`--name my-site`	컨테이너 별명. 안 주면 이상한 영어 이름이 자동으로 붙어요.
`my-web-server`	어떤 이미지로 만들지 (맨 마지막에!)

실제 실행 결과 (긴 영어+숫자 = 컨테이너 ID)

```
a30b0e4ea98aa565b7df5b7a3bca67691bcfce7dab153112bfd32bdda81d945f
```

"이름이 이미 사용 중" 에러가 뜬다면?

`docker: Error response from daemon: Conflict. The container name "/my-site" is already in use`

→ 예전 것을 지우고 다시 하세요 : `docker rm -f my-site` 입력 후 위 run 명령 다시 실행

6-4. 실행 중인지 확인

입력

```
docker ps
```

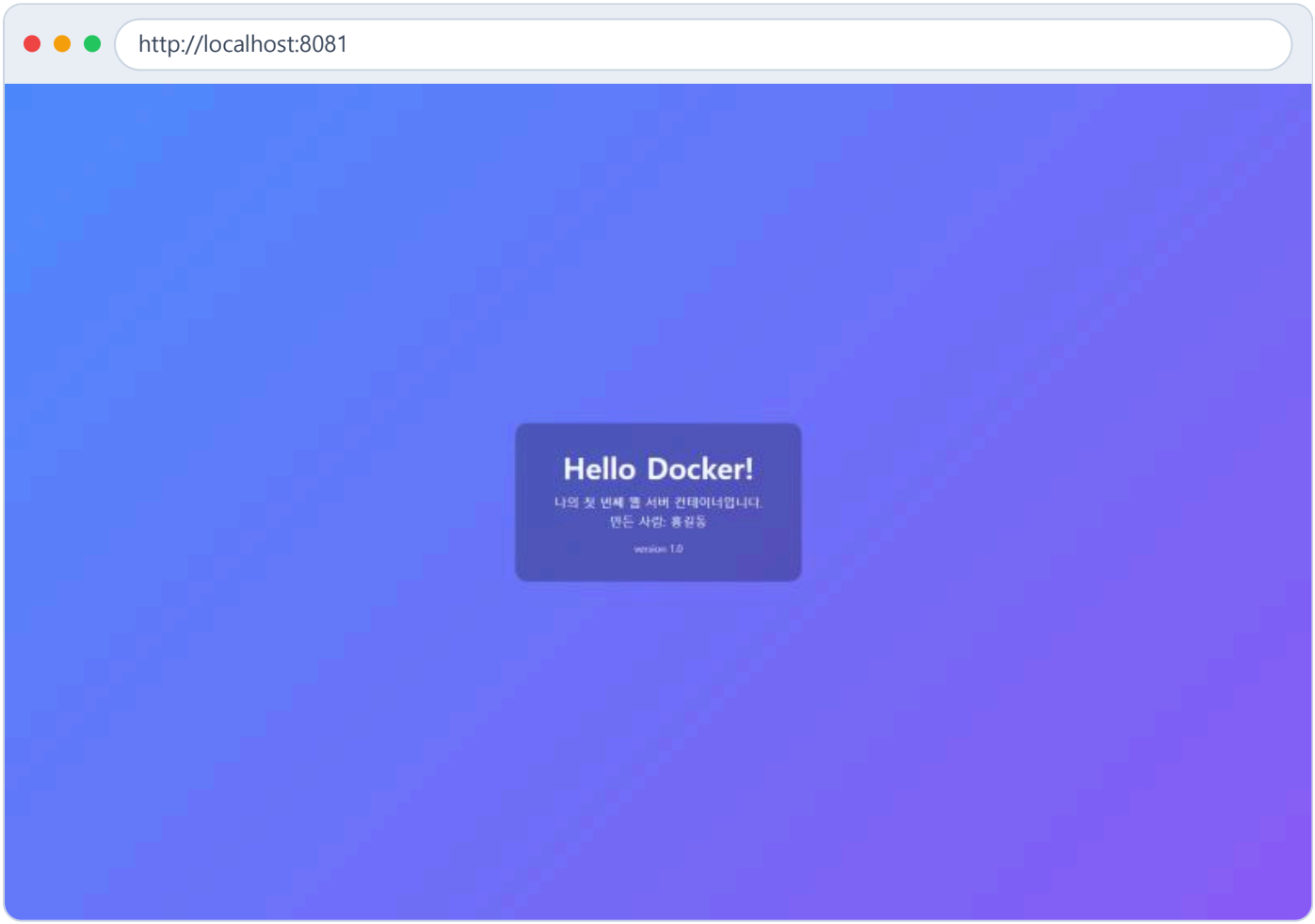
실제 실행 결과

```
CONTAINER ID   IMAGE           COMMAND                  STATUS         PORTS
NAMES
a30b0e4ea98a   my-web-server   "/docker-entrypoint.…"   Up 1 second    0.0.0.0:8081->80/tcp, [::]:8081-
>80/tcp   my-site
00b68ccfb62e   nginx           "/docker-entrypoint.…"   Up 3 seconds   0.0.0.0:8080->80/tcp, [::]:8080-
>80/tcp   my-final-web
```

성공 확인 : STATUS가 **Up**, PORTS에 **0.0.0.0:8081->80/tcp** 가 보이면 OK. **Exited** 라고 적혀 있으면 꺼진 거예요 → `docker logs my-site` 로 이유를 봅니다.

6-5. 브라우저로 접속! (증거 사진 찍는 단계)

1. 크롬(또는 엣지)을 엽니다.
2. 주소창에 **http://localhost:8081** 을 치고 Enter.
3. 내가 만든 화면이 나오면 성공! **주소창이 보이게** 캡처하세요 (**Windows 키 + Shift + S**).
4. 캡처한 사진을 `C:\study\my-web-server\docs\images\port-8081.png` 로 저장합니다.



▲ 이렇게 나오면 **포트 매핑 접속 성공 증거** 완료 (실제 http://localhost:8081 접속 화면)

터미널로도 확인 가능 : curl.exe -s -i http://localhost:8081/ (실제 결과)

```
HTTP/1.1 200 OK
Server: nginx/1.31.8
Date: Mon, 02 Nov 2026 16:06:51 GMT
Content-Type: text/html
Content-Length: 995
Connection: keep-alive

<!DOCTYPE html>
<html lang="ko"> ...
```

맨 윗줄 **HTTP/1.1 200 OK** 가 "성공"이라는 뜻이에요. 404면 파일을 못 찾은 거예요.

localhost 가 뭐예요? "내 컴퓨터 자신"이라는 뜻의 주소예요. `127.0.0.1` 과 같은 말이에요.

7 도커 운영·검증 명령 로그 모으기

목표 : 과제에서 요구하는 운영 명령을 한 번씩 실행하고 결과를 복사해 둔다.

7-1. 꼭 해야 하는 6가지

한 줄씩 실행하고 결과를 메모장에 복사해 두세요

```
# ① 이미지 목록
docker images

# ② 모든 컨테이너 (꺼진 것 포함) ← -a 가 핵심
docker ps -a

# ③ 컨테이너가 남긴 일기장(로그)
docker logs my-site

# ④ 지금 얼마나 힘들게 일하는지 (한 번만 찍고 끝)
docker stats --no-stream

# ⑤ 컨테이너 자세한 정보 중 상태만 뽑기
docker inspect my-site --format "{{.State.Status}}"

# ⑥ 상자 안에 들어가 보기 (나올 때는 exit)
docker exec -it my-site sh
```

③ docker logs my-site 실제 실행 결과 (일부)

```
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/11/02 16:06:48 [notice] 1#1: nginx/1.31.8
2026/11/02 16:06:48 [notice] 1#1: OS: Linux 6.18.54.1-microsoft-standard-WSL2
2026/11/02 16:06:48 [notice] 1#1: start worker processes
172.17.0.1 - - [02/Nov/2026:16:06:51 +0000] "GET / HTTP/1.1" 200 995 "-" "curl/8.21.0" "-"
```

맨 아랫줄 **"GET / HTTP/1.1" 200** 은 "누가 홈페이지를 봤고(GET), 잘 보여줬다(200)"는 뜻이에요. 브라우저로 접속한 다음에 logs를 찍으면 이 줄이 생깁니다. **200 = 성공, 404 = 파일 없음**.

④ docker stats --no-stream 실제 실행 결과

```
CONTAINER ID   NAME          CPU %   MEM USAGE / LIMIT     MEM %   NET I/O           BLOCK I/O     PID
S
a30b0e4ea98a   my-site       0.00%   15.13MiB / 15.18GiB   0.10%   1.46kB / 1.74kB   0B / 8.19kB   17
00b68ccfb62e   my-final-web  0.00%   13.98MiB / 15.18GiB   0.09%   1.38kB / 126B     0B / 12.3kB   17
```

⑤ docker inspect 실제 실행 결과

```
running
```

⑥ 상자 안에 들어간 모습 (실제 결과)

```
# ls /usr/share/nginx/html
50x.html
index.html
# exit
```

성공 확인 : 상자 안에 **index.html** 이 보이면, Dockerfile의 COPY가 제대로 된 거예요!

7-2. 컨테이너 껐다 켜기 연습

입력

```
docker stop my-site
docker ps -a --filter name=my-site
docker start my-site
```

실제 실행 결과

```
my-site

CONTAINER ID   IMAGE           COMMAND                  STATUS                          NAMES
a30b0e4ea98a   my-web-server   "/docker-entrypoint.…"   Exited (0) Less than a second ago   my-site

my-site
```

성공 확인 : 정지하면 STATUS가 **Exited (0)** 으로 바뀌고 PORTS가 사라져요. **(0)** 은 "문제없이 잘 꺼졌다"는 뜻입니다.

stop은 "잠깐 재우기"(내용 보존), **rm**은 "버리기"(사라짐)예요. 헷갈리지 마세요!

8 바인드 마운트 — 고치면 바로 바뀌게 하기

목표 : 내 컴퓨터 `app` 폴더를 컨테이너에 연결해서, 파일을 고치면 **다시 빌드하지 않아도** 웹페이지가 바뀌는 걸 확인한다.

왜 필요해요? STEP 6 방식은 **복사(COPY)**라서, 글자 하나만 고쳐도 **build를 다시** 해야 해요. 너무 귀찮죠. 바인드 마운트는 상자에 **창문**을 달아서 내 폴더를 직접 보게 하는 거예요. 고치면 **새로고침만** 하면 끝!

8-1. 창문을 달고 실행 (포트 8083)

입력 (한 줄입니다. 띄어쓰기 주의!)

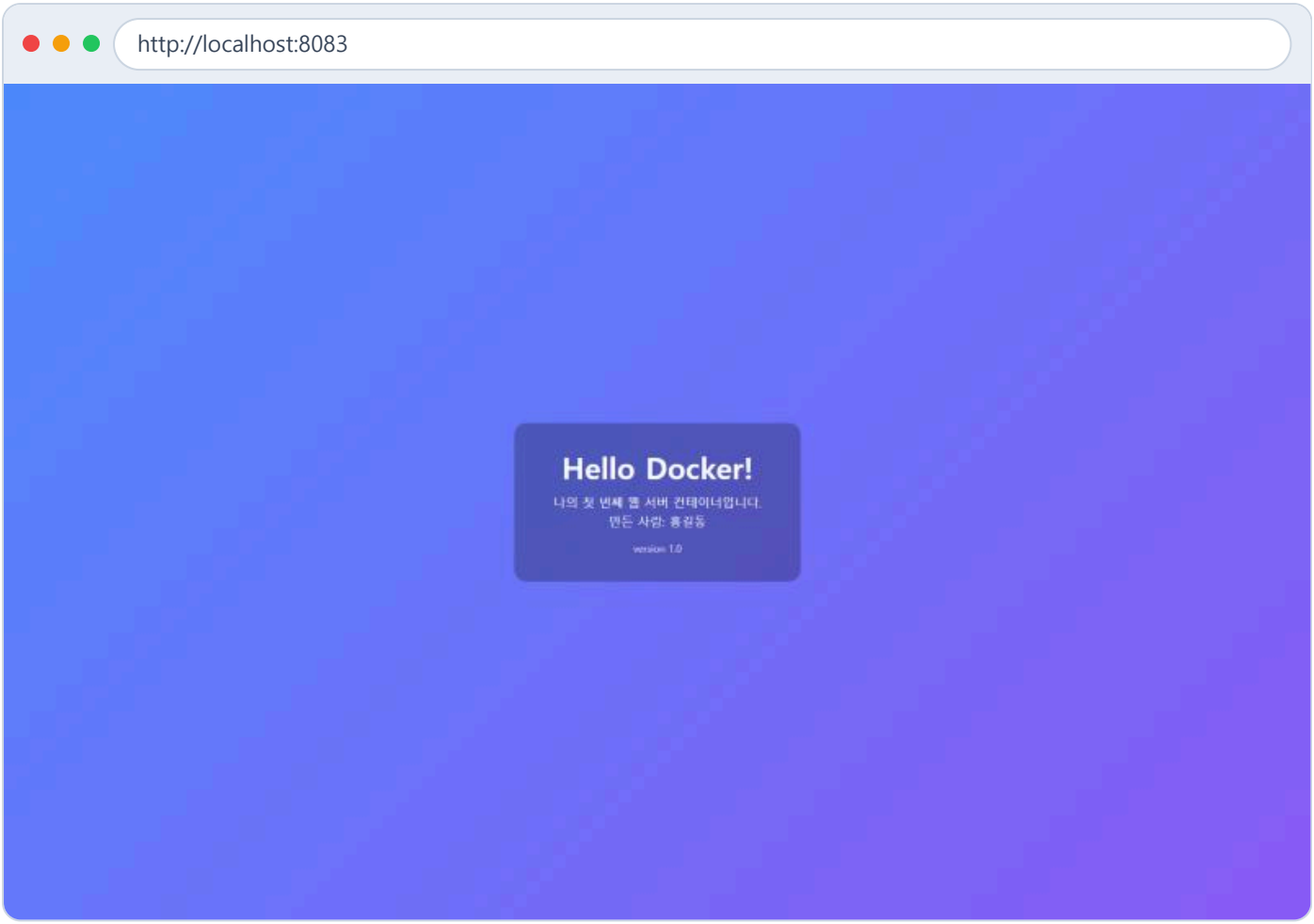
```
docker run -d -p 8083:80 --name live-site -v "C:\study\my-web-server\app:/usr/share/nginx/html" my-web-server
```

조각	뜻
`-v "왼쪽:오른쪽"`	왼쪽(**내 컴퓨터 폴더**)을 오른쪽(**상자 안 폴더**)에 겹쳐 놓기
왼쪽 `C:\study\my-web-server\app`	내 index.html 이 들어있는 진짜 폴더 (**전체 경로**로 써야 해요)
오른쪽 `/usr/share/nginx/html`	nginx가 손님에게 보여주는 폴더

폴더 안에서 실행할 땐 `-v "${PWD}\app:/usr/share/nginx/html"` 처럼 줄여 쓸 수도 있어요. (PowerShell 전용)

8-2. 바뀌기 전 화면 보기

브라우저에서 **http://localhost:8083** 접속 → 캡처 (이게 **"변경 전"** 증거)



▲ 변경 **전** : 파란 배경 / Hello Docker! / version 1.0 (실제 화면)

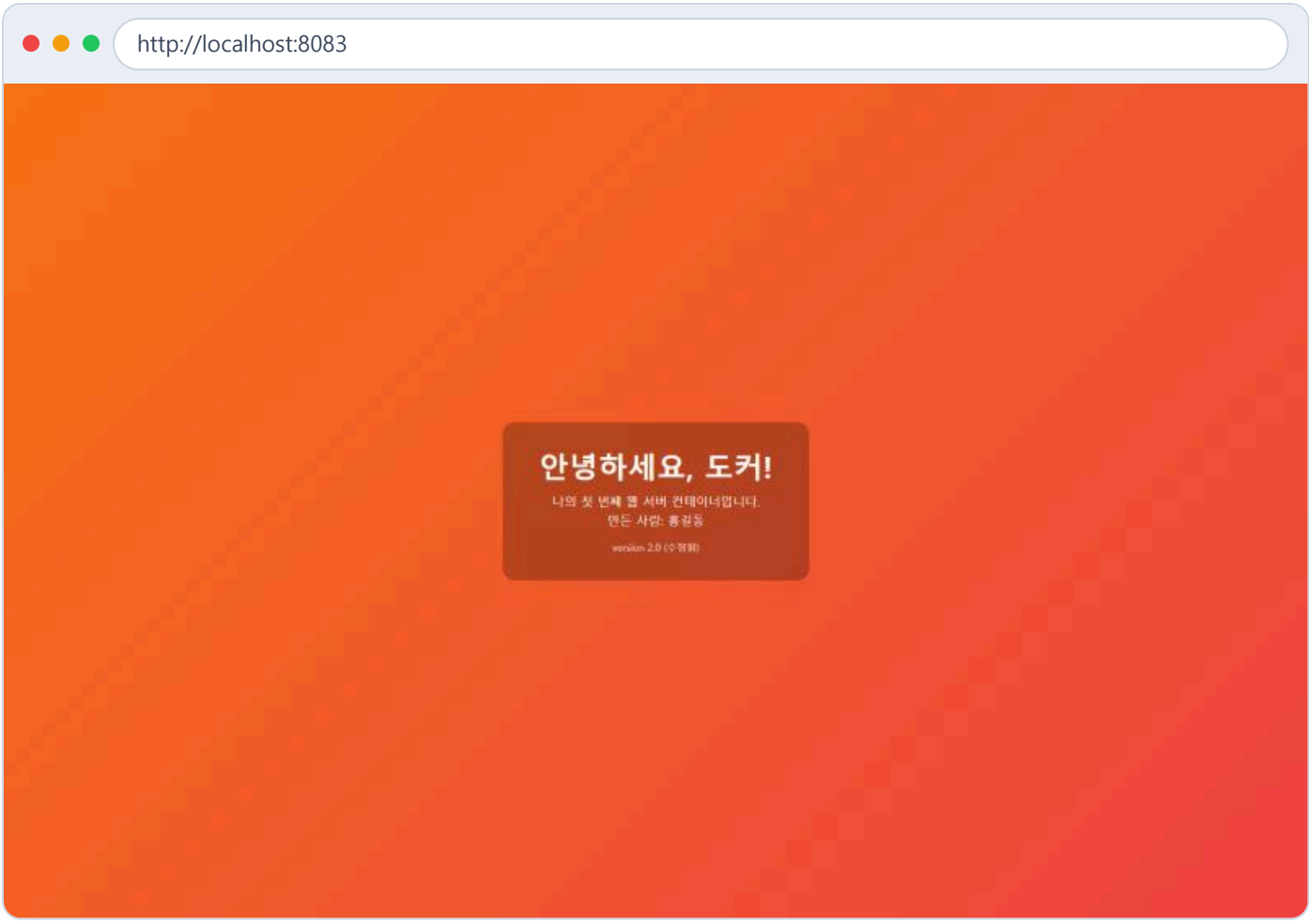
8-3. 내 컴퓨터에서 파일 고치기

`C:\study\my-web-server\app\index.html` 을 열어 세 군데를 바꾸고 **저장(Ctrl+S)** 합니다.

이렇게 바꿔 보세요

```
<h1>Hello Docker!</h1>          →  <h1>안녕하세요, 도커!</h1>
version 1.0                     →  version 2.0 (수정됨)
linear-gradient(135deg, #4f8cff, #8b5cf6)  →  linear-gradient(135deg, #f97316, #ef4444)
```

8-4. 브라우저 새로고침 (Ctrl + F5)



▲ 변경 **후** : 빌드를 다시 하지 않았는데도 주황 배경 / 안녕하세요, 도커! / version 2.0 으로 바뀜 (실제 화면)

같은 순간에 8081(COPY로 구운 컨테이너)을 보면? — 실제 비교 결과

```
[8083] 바인드 마운트  →  <h1>안녕하세요, 도커!</h1>   version 2.0 (수정됨)   주황색
[8081] COPY 로 구움   →  <h1>Hello Docker!</h1>     version 1.0           파란색
```

이게 핵심이에요! 똑같은 `my-web-server` 이미지로 만든 두 컨테이너인데, 8083은 **내 폴더를 실시간으로 보고 있어서** 바뀌었고, 8081은 **빌드할 때 복사해 둔 사진**을 보여주니까 그대로입니다.

성공 확인 : `docker build` 를 **한 번도 다시 하지 않았는데** 화면이 바뀌면 바인드 마운트 성공! 변경 전/후 캡처 2장을 `docs/images/bind-before.png` , `bind-after.png` 로 저장하세요.

안 바뀌어요! → ① 저장(Ctrl+S)을 안 했거나 ② 브라우저 캐시 때문이에요. **Ctrl + F5**(강력 새로고침)를 누르세요. 그래도 안 되면 경로가 틀린 것: `docker exec live-site ls /usr/share/nginx/html` 를 쳐서 index.html 이 보이는지 확인하세요.

8-5. 실습 끝났으면 되돌리기

index.html 을 원래대로(Hello Docker! / version 1.0 / 파란색) 고쳐 놓고 저장합니다.

입력

```
docker rm -f live-site
```

9 도커 볼륨 — 컨테이너를 지워도 안 사라지게

목표 : 볼륨을 만들어 파일을 저장하고, **컨테이너를 삭제한 뒤에도** 그 파일이 남아있는 걸 확인한다.

왜 필요해요? 컨테이너는 **지우면 안의 내용이 전부 사라져요**(마치 모래성처럼). 그래서 게임 세이브파일이나 사진처럼 **지키고 싶은 것**은 컨테이너 밖 **볼륨(보관함)**에 넣어 둡니다.

9-1. 볼륨 만들기

입력

```
docker volume create my-volume
docker volume ls
```

실제 실행 결과

```
my-volume

DRIVER    VOLUME NAME
local     my-volume
```

9-2. 볼륨을 연결해서 컨테이너 실행 (포트 8082)

입력

```
docker run -d -p 8082:80 --name vol-test -v my-volume:/usr/share/nginx/html/data my-web-server
```

바인드 마운트와 다른 점 : `-v` 왼쪽이 **폴더 경로**가 아니라 **볼륨 이름**(my-volume)이에요. 도커가 알아서 자기 창고에 보관해 줍니다.

9-3. 볼륨 안에 메모 남기기

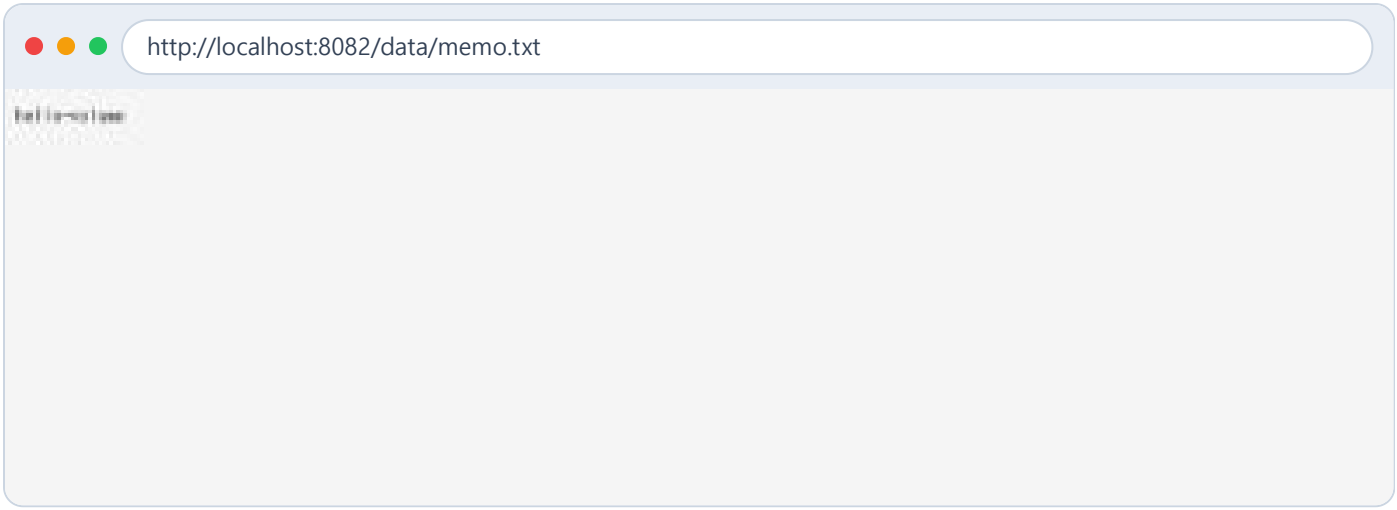
입력

```
docker exec vol-test sh -c "echo hello-volume > /usr/share/nginx/html/data/memo.txt"
docker exec vol-test cat /usr/share/nginx/html/data/memo.txt
```

실제 실행 결과

```
hello-volume
```

브라우저로도 확인할 수 있어요 → **http://localhost:8082/data/memo.txt**



▲ 볼륨 안에 넣은 메모가 웹으로 보임 (실제 화면)

9-4. ★ 컨테이너를 통째로 삭제! (무서워도 괜찮아요)

입력

```
docker rm -f vol-test
docker ps -a
```

실제 실행 결과 (목록이 비어 있음 = 완전히 사라짐)

```
vol-test

CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

`docker ps -a` 목록에서 **vol-test 가 사라진 것**을 확인하고 캡처하세요. (= **삭제 전/후 비교의 '후'**)

9-5. 새 컨테이너를 만들어 같은 볼륨 붙이기

입력

```
docker run -d -p 8082:80 --name vol-test2 -v my-volume:/usr/share/nginx/html/data my-web-server
docker exec vol-test2 cat /usr/share/nginx/html/data/memo.txt
```

실제 실행 결과 — 컨테이너를 지웠는데도 그대로!

```
dc1c904096744e81ee4c3a74ba7fe1814dc69862fd918184f1f00fcb5404eafa
hello-volume
```

성공 확인 : 컨테이너를 **완전히 지웠는데도** memo.txt 가 그대로 살아있으면 **볼륨 영속성 증명 완료!** 브라우저 **http://localhost:8082/data/memo.txt** 에서도 똑같이 보입니다. 이 화면도 캡처하세요.

9-6. 볼륨이 어디 저장되는지 구경하기

입력

```
docker volume inspect my-volume
```

실제 실행 결과


```
[
    {
        "CreatedAt": "2026-11-02T16:06:57Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/my-volume/_data",
        "Name": "my-volume",
        "Options": null,
        "Scope": "local"
    }
]
```


Mountpoint 가 볼륨이 실제로 저장되는 자리예요. 내 C 드라이브가 아니라 도커가 WSL 안에 만들어 둔 창고 주소라서, 탐색기로는 바로 안 보입니다.

9-7. 비교 표 (README에 그대로 넣기 좋아요)

구분	바인드 마운트	볼륨
적는 법	`-v "C:\...\app:/경로"`	`-v my-volume:/경로`
저장 위치	내 컴퓨터 폴더	도커가 관리하는 창고
좋은 점	고치면 바로 반영(개발할 때 최고)	컨테이너를 지워도 안 사라짐(데이터 보관 최고)
이번 실습 증거	8083 변경 전/후 화면	8082 컨테이너 삭제 전/후 memo.txt

10 Git 설정하고 내 작업 저장하기

목표 : 이름·이메일·기본 브랜치를 설정하고, 내 폴더를 Git 저장소로 만들어 첫 커밋을 남긴다.

Git이 뭐예요? 게임의 **세이브(저장) 기능**이에요. "여기까지 잘 됐다!" 싶을 때 저장해 두면, 나중에 망가져도 그 지점으로 돌아갈 수 있어요. 그 저장을 **커밋(commit)** 이라고 부릅니다.

10-1. 내 이름표 달기 (컴퓨터에 한 번만 하면 됨)

입력 (따옴표 안은 내 정보로 바꾸기)

```
git config --global user.name "Hong Gildong"
git config --global user.email "mygithub@example.com"
git config --global init.defaultBranch main
git config --global --list
```

출력 예시

```
user.name=Hong Gildong
user.email=mygithub@example.com
init.defaultBranch=main
```

이메일 주의! GitHub에 가입한 이메일과 같게 쓰세요. 다르면 커밋에 내 얼굴(프로필)이 안 붙어요. 그리고 **비밀번호·토큰은 절대 명령어에 적지 않습니다.**

main 이 뭐예요? 저장 기록이 쌓이는 **큰 줄기**의 이름이에요. 예전엔 master 였는데 지금은 **main** 을 씁니다.

10-2. 내 폴더를 Git 저장소로 만들기

입력

```
cd C:\study\my-web-server
git init
git status
```

실제 실행 결과

```
Initialized empty Git repository in C:/study/my-web-server/.git/

?? .gitignore
?? Dockerfile
?? README.md
?? app/
?? docs/
```

> **Untracked** = "아직 저장 안 한 새 파일"이라는 뜻이에요.

10-3. 올리면 안 되는 파일 걸러내기 (.gitignore)

`C:\study\my-web-server\.gitignore` 파일을 만들고 아래 내용을 적습니다.

.gitignore

```
# 로그 / 임시 파일
*.log
tmp/
.DS_Store
Thumbs.db

# 비밀 정보 (토큰, 비밀번호) 절대 올리지 않기
.env
secret*
*.key
```

> **개인정보 보호 규칙** : 아이디·비밀번호·GitHub 토큰·전화번호·집 주소는 **코드에도, README에도, 캡처 사진에도** 넣지 마세요. 캡처할 때 화면에 이메일이 보이면 **모자이크** 하세요.

10-4. 첫 번째 저장(커밋)

입력

```
git add .
git commit -m "feat: 첫 웹 서버 컨테이너 과제 제출"
git log --oneline
git branch --show-current
```

실제 실행 결과

```
[master (root-commit) e1578d6] feat: 첫 웹 서버 컨테이너 과제 제출
 6 files changed, 196 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 Dockerfile
 create mode 100644 README.md
 create mode 100644 app/index.html

e1578d6 feat: 첫 웹 서버 컨테이너 과제 제출
master
```

> **어? main 이 아니라 master 라고 나왔어요!** 이건 10-1에서 `git config --global init.defaultBranch main` 을 **안 한 상태**에서 `git init` 했기 때문이에요. (이 책을 만들며 실제로 재현한 상황입니다.) 당황하지 말고 다음 한 줄이면 고쳐집니다.

입력

```
git branch -M main
git branch --show-current
```

실제 실행 결과

```
main
```

명령	뜻
`git add .`	바뀐 파일 **전부**를 장바구니에 담기 (점 = 전부)
`git commit -m "글"`	장바구니를 **저장**하고 설명 붙이기
`git log --oneline`	저장 기록 목록 보기

성공 확인 : `git log --oneline` 에 내 커밋 한 줄이 보이고, 브랜치가 **main** 이면 OK.

경고가 떠요 : LF will be replaced by CRLF → 에러가 아닙니다. 윈도우와 리눅스의 '줄바꿈 글자'가 달라서 알려주는 것뿐이니 그냥 진행하세요.

11 GitHub에 올리기 + VSCode 연동

목표 : GitHub에 저장소를 만들고, VSCode에서 로그인해서 내 작업을 인터넷에 올린다(push).

11-1. GitHub에 빈 저장소 만들기

1. **github.com** 접속 → 로그인 (계정이 없으면 Sign up)
2. 오른쪽 위 **+** → **New repository**
3. Repository name : **my-web-server**
4. Description : **Docker 웹서버 구축 미션**
5. **Public** 선택 (선생님이 링크로 볼 수 있어야 하니까요)
6. **Add a README file 체크 해제** ← 체크하면 push할 때 충돌이 나요!
7. **Create repository** 클릭

11-2. 방법 A — 터미널로 올리기

입력 (<내아이디> 부분만 내 걸로 바꾸기)

```
cd C:\study\my-web-server
git branch -M main
git remote add origin https://github.com/<내아이디>/my-web-server.git
git remote -v
git push -u origin main
```

git remote -v 출력 예시 (실제 검증 결과)

```
origin  https://github.com/mygithub/my-web-server.git (fetch)
origin  https://github.com/mygithub/my-web-server.git (push)
```

처음 push 하면 **로그인 창**이 떠요. **Sign in with your browser** 를 눌러 GitHub 계정으로 로그인하면 끝!

push 성공 출력 예시

```
Enumerating objects: 9, done.
Writing objects: 100% (9/9), 2.31 KiB | 1.15 MiB/s, done.
To https://github.com/mygithub/my-web-server.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

성공 확인 : **[new branch] main -> main** 이 보이고, 브라우저에서 내 저장소를 새로고침하면 파일들이 올라가 있어요.

11-3. 방법 B — VSCode로 올리기 (더 쉬움, 과제 요구사항)

1. VSCode 실행 → **파일 → 폴더 열기** → `C:\study\my-web-server` 선택
2. 왼쪽 맨 아래 **사람 모양 아이콘(계정)** 클릭 → **Sign in with GitHub** → 브라우저에서 **Authorize**

3. 왼쪽 **소스 제어** 아이콘(가지 세 개 모양) 클릭
4. 메시지 칸에 커밋 설명을 쓰고 ✓ **커밋** 클릭
5. **Publish Branch**(또는 **Sync Changes**) 클릭 → **Public** 선택
6. "Successfully published" 알림이 뜨면 완료!

> **증거 캡처** : ① VSCode 왼쪽 아래에 **내 GitHub 아이디**가 보이는 화면, ② 소스 제어 패널에 커밋 기록이 보이는 화면, ③ GitHub 저장소 페이지 화면 — 이렇게 3장을 `docs/images/` 에 저장하세요. (아이디는 괜찮지만 **토큰·비밀번호 화면은 금지**)

11-4. 다음부터 저장하는 방법 (3줄만 기억!)

입력

```
git add .
git commit -m "docs: README 보강"
git push
```

12 README.md 쓰기 (제일 중요한 점수!)

목표 : "이 글만 읽어도 내가 뭘 했는지 전부 알 수 있는" 기술 문서를 쓴다.

README가 뭐예요? 저장소에 들어오면 **맨 처음 보이는 설명서**예요. 장난감 상자의 설명서처럼, 처음 보는 사람도 따라할 수 있게 적어야 해요.

12-1. 꼭 들어가야 하는 7가지

- [ ] 프로젝트 개요 (이 과제가 뭔지 3~5줄)
- [ ] 실행 환경 (OS / 터미널 / Docker 버전 / Git 버전)
- [ ] 수행 항목 체크리스트 (터미널·권한·Docker·Dockerfile·포트·볼륨·Git·GitHub)
- [ ] 검증 방법 (어떤 명령으로 무엇을 확인했는지) + 결과 사진 링크
- [ ] 포트 매핑 접속 증거 (주소창 보이는 캡처)
- [ ] 바인드 마운트 전/후 + 볼륨 삭제 전/후 증거
- [ ] **트러블슈팅 2건 이상** (문제 → 원인 가설 → 확인 → 해결)

12-2. 복사해서 쓰는 README.md 뼈대

C:₩study₩my-web-server₩README.md

```
# 🐳 나의 첫 웹 서버 (Docker 구축 미션)

## 1. 프로젝트 개요
Windows 11 + Docker Desktop 환경에서 nginx 기반 웹 서버 컨테이너를 직접 만들고,
포트 매핑 / 바인드 마운트 / 볼륨 영속성을 확인한 뒤 GitHub에 제출한 과제입니다.

## 2. 실행 환경
| 항목 | 값 |
|---|---|
| OS | Windows 11 Pro (26200) |
| 터미널 | Windows PowerShell |
| Docker | Docker version 27.3.1 |
| Git | git version 2.54.0.windows.1 |
| 에디터 | Visual Studio Code |

## 3. 폴더 구조
my-web-server/
├─ app/index.html      # 웹 서버 소스코드
├─ Dockerfile          # 이미지 레시피
├─ .gitignore
├─ docs/images/        # 증거 스크린샷
└─ README.md

## 4. 수행 항목 체크리스트
- [x] 터미널 기본 조작 (pwd / cd / mkdir / dir / type)
- [x] 권한 확인 (whoami / icacls / 관리자 권한 실행)
- [x] Docker 설치 점검 (docker --version / docker info / hello-world)
- [x] Dockerfile 작성 및 이미지 빌드 (my-web-server)
- [x] 포트 매핑 실행 및 브라우저 접속 (8081 → 80)
- [x] 운영 명령 (images / ps -a / logs / stats / inspect / exec)
- [x] 바인드 마운트 반영 확인 (8083)
- [x] 볼륨 영속성 확인 (8082, 컨테이너 삭제 전/후)
- [x] Git 설정 및 커밋 (main 브랜치)
- [x] VSCode GitHub 로그인 및 저장소 연동

## 5. 실행 방법
    git clone https://github.com/<내아이디>/my-web-server.git
    cd my-web-server
    docker build -t my-web-server .
    docker run -d -p 8081:80 --name my-site my-web-server
    # 브라우저에서 http://localhost:8081

## 6. 검증 방법과 결과
| 확인할 것 | 사용한 명령 | 기대 결과 | 증거 |
|---|---|---|---|
| 이미지 생성 | `docker images` | my-web-server 목록에 있음 | ![](docs/images/images.png) |
| 컨테이너 실행 | `docker ps` | STATUS=Up, 8081→80 | ![](docs/images/ps.png) |
| 포트 접속 | 브라우저 http://localhost:8081 | 내 페이지 표시 | ![](docs/images/port-8081.png) |
| 접속 로그 | `docker logs my-site` | "GET / HTTP/1.1" 200 | ![](docs/images/logs.png) |
```

```
| 자원 사용 | `docker stats --no-stream` | CPU/MEM 표시 | ![](docs/images/stats.png) |
| 바인드 마운트 | 호스트 index.html 수정 후 새로고침 | 재빌드 없이 변경 반영 | ![](docs/images/bind-before.png) ![](docs/images/bind-after.png) |
| 볼륨 영속성 | `docker rm -f vol-test` 후 재실행 | memo.txt 그대로 유지 | ![](docs/images/volume-before.png) ![](docs/images/volume-after.png) |
| GitHub 연동 | VSCode 로그인 / `git push` | 저장소에 코드 업로드 | ![](docs/images/vscode-github.png) |

## 7. 트러블슈팅
### ① 포트 충돌 (port is already allocated)
- **문제**: `docker run -p 8080:80` 실행 시 `Bind for 0.0.0.0:8080 failed: port is already allocated`
- **원인 가설**: 8080 포트를 이미 다른 컨테이너(my-final-web)가 사용 중
- **확인**: `docker ps` 실행 → PORTS에 `0.0.0.0:8080->80/tcp` 발견
- **해결**: 비어 있는 8081 포트로 변경하여 재실행 → 정상 접속

### ② 컨테이너 이름 중복 (name is already in use)
- **문제**: `--name my-site` 로 실행 시 Conflict 에러
- **원인 가설**: 같은 이름의 이전 컨테이너가 남아 있음
- **확인**: `docker ps -a` 에서 Exited 상태의 my-site 발견
- **해결**: `docker rm -f my-site` 로 삭제 후 재실행 → 정상 기동

## 8. 뒷정리 명령
    docker rm -f my-site live-site vol-test2
    docker volume rm my-volume
    docker image rm my-web-server
```

> README.md 도 UTF-8 로 저장해야 한글이 안 깨져요. VSCode는 기본이 UTF-8이라 그냥 저장하면 됩니다.

13 제출 전 마지막 점검표

하나씩 ☐ 에 체크하면서 확인하세요. 전부 체크되면 제출 완료!

A. 저장소

- ☐ GitHub 저장소가 **Public** 으로 공개되어 있다
- ☐ 저장소 **링크 하나만** 열어도 코드·README·사진이 전부 보인다
- ☐ 아이디/비밀번호/토큰이 들어간 파일이나 사진이 **없다**

B. 코드와 파일

- ☐ `app/index.html` 이 올라가 있다
- ☐ `Dockerfile` 이 올라가 있다 (확장자 없음)
- ☐ `.gitignore` 가 있다
- ☐ `docs/images/` 안에 증거 사진들이 있다

C. README 내용

- ☐ 프로젝트 개요가 있다
- ☐ 실행 환경 표(OS/터미널/Docker/Git 버전)가 있다
- ☐ 수행 항목 체크리스트 8줄이 있다
- ☐ 검증 방법 표(명령 + 기대결과 + 사진)가 있다
- ☐ 트러블슈팅이 **2건 이상** 있고, 문제→원인 가설→확인→해결 4단계로 적혀 있다

D. 증거 사진

- ☐ 터미널에서 `docker --version`, `docker info` 결과
- ☐ `docker images` / `docker ps -a` 결과
- ☐ `docker logs` / `docker stats` 결과
- ☐ 브라우저 **주소창이 보이는** localhost:8081 접속 화면
- ☐ 바인드 마운트 **변경 전 / 변경 후** 2장
- ☐ 볼륨 **컨테이너 삭제 전 / 삭제 후** 2장
- ☐ VSCode GitHub 로그인 + 저장소 연동 화면

14 에러가 났을 때 보는 응급처치 8가지

에러 메시지를 **그대로 읽는 것**이 해결의 90%예요. 아래에서 비슷한 문장을 찾아보세요.

① docker 명령을 못 찾아요

에러

```
docker : The term 'docker' is not recognized as the name of a cmdlet...
```

- **원인 가설** : Docker Desktop이 설치 안 됐거나, 설치 후 터미널을 다시 안 열었음
- **확인** : 작업표시줄에 고래 아이콘이 있는지 / `Get-Command docker` 실행
- **해결** : Docker Desktop 설치 → **컴퓨터 재부팅** → 터미널 새로 열고 `docker --version`

② 도커 엔진에 연결이 안 돼요

에러

```
error during connect: ... open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
```

- **원인 가설** : Docker Desktop 프로그램이 꺼져 있음
- **확인** : 고래 아이콘 색깔 / Docker Desktop 창의 좌측 하단 상태 글씨
- **해결** : Docker Desktop 실행 후 **Engine running** 될 때까지 1~2분 기다리고 다시 실행

③ 포트가 이미 사용 중이에요

실제로 재현한 에러 메시지

```
docker: Error response from daemon: failed to set up container networking:
driver failed programming external connectivity on endpoint conflict-test
(4dc7cf32b5494cda2d330889d69eedd4730dd95bda14073cfaf885d164b34818):
Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
```

> 글이 길지만 **맨 아랫줄만** 보면 돼요 : **port is already allocated** = "그 번호는 이미 찼어요".

- **원인 가설** : 그 번호를 다른 컨테이너/프로그램이 쓰고 있음
- **확인** : `docker ps` 의 PORTS 열 확인, 또는 `netstat -ano | findstr :8080`
- **해결** : ⓐ 다른 번호 사용(`-p 8081:80`) 또는 ⓑ 쓰던 컨테이너 정지(`docker stop my-final-web`)

④ 컨테이너 이름이 중복이에요

실제로 재현한 에러 메시지

```
docker: Error response from daemon: Conflict. The container name "/my-site"
is already in use by container
"a30b0e4ea98aa565b7df5b7a3bca67691bcfce7dab153112bfd32bdda81d945f".
You have to remove (or rename) that container to be able to reuse that name.
```

- **원인 가설** : 예전에 만든 같은 이름 컨테이너가 남아 있음 (꺼져 있어도 이름은 차지함)
- **확인** : `docker ps -a` 로 전체 목록 보기
- **해결** : `docker rm -f my-site` 후 다시 run

⑤ 빌드할 때 Dockerfile을 못 찾아요

에러

```
ERROR: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
```

- **원인 가설** : 다른 폴더에서 빌드했거나, 파일 이름이 `Dockerfile.txt` 로 저장됨
- **확인** : `pwd` 로 위치 확인, `dir` 로 이름 확인(확장자 숨김 해제 : 탐색기 → 보기 → 파일 확장명 체크)
- **해결** : 올바른 폴더로 `cd` 하고, 이름을 **Dockerfile** 로 바꾼 뒤 `docker build -t my-web-server .` (**마지막 점 필수**)

⑥ 페이지가 404거나 옛날 화면이 나와요

- **원인 가설** : ⓐ index.html 을 고친 뒤 **재빌드를 안 함** ⓑ 브라우저가 옛날 화면을 기억(캐시)
- **확인** : `docker exec my-site ls /usr/share/nginx/html` → index.html 이 있는지 / `docker logs my-site` 에 404 기록 확인
- **해결** : `docker rm -f my-site` → `docker build -t my-web-server .` → 다시 run → 브라우저에서 **Ctrl+F5**

⑦ 한글이 □□□ 로 깨져요

- **원인 가설** : HTML에 `<meta charset="UTF-8">` 가 없거나, 파일이 ANSI로 저장됨
- **확인** : 메모장 → 다른 이름으로 저장 → 아래쪽 **인코딩** 표시 확인
- **해결** : 인코딩을 **UTF-8** 로 저장하고 `docker build` 다시 → 새로고침

> **이 책을 만들면서 실제로 겪은 일!** PowerShell 스크립트로 index.html을 고쳤더니 한글이 **?댐꿨?샗쫽??** 처럼 깨졌습니다. 원인은 **Windows PowerShell 5.1이 BOM 없는 UTF-8 파일을 ANSI로 읽는 것**이었어요. 메모장·VSCode로 직접 고칠 때는 괜찮습니다. 스크립트로 파일을 고칠 거라면 저장할 때 꼭 UTF-8을 지정하세요.

⑧ 설치는 됐는데 "Docker Desktop is unable to start"

실제로 겪은 에러

```
Error response from daemon: Docker Desktop is unable to start
```

- **원인 가설** : Docker Desktop은 깔렸지만 **WSL2**(리눅스 엔진)가 아직 없음
- **확인** : `wsl --version` 실행 → "Linux용 Windows 하위 시스템이 설치되어 있지 않습니다" 메시지
- **해결** : **관리자 권한** PowerShell에서 `wsl --install --no-distribution` → **재부팅** → Docker Desktop 다시 실행 → `docker info` 로 Server Version 확인

> 도커는 사실 **리눅스** 프로그램이에요. 그래서 윈도우 안에 작은 리눅스(WSL2)를 두고 그 위에서 돌아갑니다. 윈도우에서 도커를 처음 깔 때 재부팅을 한 번 하게 되는 이유예요.

15 명령어 모음집 (한 장 요약)

터미널

명령	뜻
`pwd`	지금 위치 보기
`cd C:\study\my-web-server`	폴더 이동
`mkdir app`	폴더 만들기
`dir`	목록 보기
`type Dockerfile`	파일 내용 보기
`whoami` / `icacls 파일`	내 계정 / 파일 권한 보기

도커

명령	뜻
`docker --version` / `docker info`	버전 / 상태
`docker build -t 이름 .`	이미지 굽기
`docker images`	이미지 목록
`docker run -d -p 8081:80 --name my-site 이미지`	컨테이너 실행
`docker ps` / `docker ps -a`	실행 중 / 전체 목록
`docker logs 이름`	로그 보기
`docker stats --no-stream`	자원 사용량
`docker exec -it 이름 sh`	상자 안으로 들어가기 (나올 땐 exit)
`docker stop / start / rm -f 이름`	정지 / 시작 / 삭제
`docker volume create / ls / inspect / rm`	볼륨 만들기·보기·지우기

Git

명령	뜻
`git config --global user.name "이름"`	이름 등록
`git init` / `git status`	저장소 만들기 / 상태
`git add .` / `git commit -m "글"`	담기 / 저장
`git branch -M main`	브랜치 이름 main 으로
`git remote add origin 주소`	GitHub 주소 연결
`git push -u origin main`	인터넷에 올리기

실습이 끝난 뒤 뒷정리 (선택)

입력

```
docker rm -f live-site vol-test2
docker volume rm my-volume
docker ps -a
```

my-site(8081) 와 my-final-web(8080) 은 지우지 마세요! 과제 제출 증거로 살려 둡니다.

이 책의 검증 환경

아래 환경에서 가이드의 명령 **40개를 실제로 실행**하고, 그 출력을 그대로 실었습니다. 포트 충돌·이름 중복 에러도 일부러 만들어서 받아 적은 진짜 메시지입니다.

항목	값
검증 날짜	2026-11-03
OS	Windows 11 Pro (build 26200)
Docker	Docker version 29.9.2, build a82e528 (Docker Desktop 4.97.0)
Docker Compose	v5.6.1
엔진 / 커널	WSL2 backend · Linux 6.18.54.1-microsoft-standard-WSL2
웹 서버	nginx 1.31.8 (nginx:latest)
Git	git version 2.54.0.windows.1
확인한 포트	8080 my-final-web · 8081 my-site · 8082 볼륨 · 8083 바인드 마운트

🎉 여기까지 왔다면 미션 완료! 🎉

GitHub 저장소 링크를 선생님께 제출하세요.